SIAGA API: Technical Report

The SIAGA API is the cross-border health-data interoperability layer of the ADHCRI platform (SDG 17). It exposes one harmonized schema over ASEAN health indicators, the disease forecasts, the model's driver ranking, and a lightweight life-expectancy inference endpoint.

---

1. Design goals

1. **Interoperability (SDG 17):** one comparable schema so member states can share and query health data across borders without bespoke integration.
2. **Climate resilience:** the brief calls out unreliable power and connectivity in rural facilities. The API is a single static binary with zero external dependencies, so it deploys and runs anywhere, including offline edge hardware.
3. **Edge inference:** life-expectancy prediction runs locally through a compact linear surrogate, so a facility can get a prediction without reaching a central server or a Python runtime.

2. Technology

- **Language:** Go (standard library only: net/http, encoding/json, encoding/csv).
- **Footprint:** about 8 MB static binary, no runtime dependencies.
- **Data:** loads the harmonized CSV/JSON serving files at startup into memory.
- **Concurrency:** Go's net/http serves requests concurrently by default.

3. Endpoints

Method	Path	Query / body	Returns
GET	/health	-	service status, panel row count, surrogate R2
GET	/countries	-	member states with ISO3 codes
GET	/indicators	country, indicator, from, to	harmonized panel rows
GET	/forecast	disease (tb/malaria), country	5-year forecast rows with 80% intervals
GET	/drivers	-	SHAP driver ranking, descending
POST	/predict	JSON {"features": {name: value}}	predicted life expectancy

Examples

```
# harmonized indicator query
curl "http://localhost:8080/indicators?
country=Singapore&indicator=life_expectancy&from=2012"

# five-year TB forecast for one country
curl "http://localhost:8080/forecast?disease=tb&country=Cambodia"

# driver ranking
```

```
curl "http://localhost:8080/drivers"

# edge life-expectancy inference
curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" \
  -d '{"features": {"physicians_per_1000": 1.5, "undernourished_pct": 8,
"tb_prevalence": 300, "year": 2019}}'
```

Response shapes

`GET /health`

```
{"status": "ok", "service": "siaga-api", "panel_rows": 252, "surrogate_r2": 0.961}
```

`POST /predict`

```
{"predicted_life_expectancy": 71.4, "features_used": 4, "model": "linear_surrogate",
 "note": "edge approximation of the reference GBM; see notebook for the primary model"}
```

4. The inference model

The `/predict` endpoint runs a standardized, sign-constrained linear surrogate of the reference gradient-boosting model:

```
prediction = intercept + sum over features of coef_i * (x_i - mean_i) / std_i
```

Coefficients are constrained to the same monotonic medical priors as the reference model (more staffing, immunization, and spend never lower the prediction; more disease and undernourishment never raise it), so edge predictions are always directionally sound. Absent features fall back to the training mean and contribute zero, so the endpoint degrades gracefully on partial field data. In-sample R2 is 0.96. The full gradient-boosting model remains the reference for the notebook and the dashboard simulator.

5. Authentication and rate limiting

The API enforces two protections, both in the standard library (no dependencies):

- **API-key authentication.** Every endpoint except `/health` requires a valid key, presented as `X-API-Key: <key>` or `Authorization: Bearer <key>`. Valid keys are configured via the `SIAGA_API_KEYS` environment variable (comma-separated). If it is empty, auth is disabled and a warning is logged, so local development is friction-free while production is secured by setting real keys. A missing or invalid key returns `401 Unauthorized`.
- **Per-client rate limiting.** A token-bucket limiter caps each client IP at `SIAGA_RATE_RPM` requests per minute (default 60), returning `429 Too Many Requests` with a `Retry-After` header when exceeded. `/health` is exempt so liveness probes are never throttled. Idle buckets are reaped so memory cannot grow unbounded.

Example:

```
export SIAGA_API_KEYS="team-key-1,team-key-2"
export SIAGA_RATE_RPM=120
go run .
curl -H "X-API-Key: team-key-1" http://localhost:8080/drivers
```

6. Operations and deployment

- **Run locally:** `cd siaga/api && go run .` (or `go build -o siaga-api .`).
- **Containers:** `deploy/Dockerfile.api` builds a distroless static image (a few MB); `deploy/docker-compose.yml` runs the API and dashboard together.
- **Port:** 8080 by default, override with `PORT`. **Data:** `SIAGA_DATA` (default `data/`).
- **CORS:** permissive so the browser dashboard can call it cross-origin; the auth headers are allowed.

7. Security posture and limitations

- The API is read-only except for `/predict`, a pure function with no persistence, so it holds no patient data and presents no data-exposure surface. SIAGA operates on aggregate national indicators, never individual records.
- Auth and rate limiting are implemented in the application. Production would add TLS termination, secret management for the keys, and audit logging at the gateway.
- The surrogate is an approximation intended for edge use; authoritative predictions should use the reference model in the analytics layer.